

BÁO CÁO MÃ NGUỒN - LAB 05

****Sinh viên thực hiện:**** Lê Viết Đức

****MSSV:**** 23DTHA4 / 0533

****Môn học:**** Thực hành Bảo mật thông tin nâng cao

Đường dẫn file: `lab-05\base64\decode.py`

```
import base64
import binascii

def validate_base64(s):
    """Kiểm tra sơ bộ tính hợp lệ của chuỗi Base64"""
    # Loại bỏ các ký tự khoảng trắng nếu có
    s = s.strip()

    # Độ dài chuỗi Base64 phải chia hết cho 4 (kể cả phần bù = nếu có)
    if len(s) % 4 != 0:
        return False, "Độ dài chuỗi Base64 không hợp lệ (phải chia hết cho 4).\"

    # Tập ký tự hợp lệ trong Base64
    valid_chars = set("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/=")
    if not all(char in valid_chars for char in s):
        return False, "Chuỗi chứa ký tự không hợp lệ trong bảng mã Base64.\"

    return True, \"

def main():
    print(\"=== BASE64 DECODE ===\")
    try:
        user_input = input(\"Nhập chuỗi Base64 cần giải mã: \").strip()

        # Kiểm tra tính hợp lệ trước khi giải mã
        is_valid, err_msg = validate_base64(user_input)
        if not is_valid:
            print(f\"\\n[Lỗi Định Dạng] {err_msg}\")
            return

        # Giải mã Base64
        # Chuyển chuỗi đầu vào thành bytes
        encoded_bytes = user_input.encode('utf-8')
```

```

# Giải mã
decoded_bytes = base64.b64decode(encoded_bytes)

# Chuyển bytes kết quả thành chuỗi utf-8
decoded_str = decoded_bytes.decode('utf-8')

print("\n--- Kết quả giải mã ---")
print(f"Chuỗi Base64: {user_input}")
print(f"Chuỗi ban đầu: {decoded_str}")
print("-----")

except (binascii.Error, ValueError) as e:
    print(f"\n[Lỗi Giải Mã] Chuỗi Base64 không hợp lệ hoặc sai định dạng padding: {e}")
except UnicodeDecodeError:
    print("\n[Lỗi Giải Mã] Chuỗi giải mã thành công dạng bytes nhưng không thể hiển thị dưới dạng văn bản UTF-8 (có thể đây là dữ liệu nhị phân).")
except Exception as e:
    print(f"\n[Lỗi] Có lỗi hệ thống xảy ra: {e}")

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab-05\base64\encode.py`

```

import base64

def main():
    print("=== BASE64 ENCODE ===")
    try:
        user_input = input("Nhập chuỗi văn bản cần mã hóa: ")

        # Chuyển chuỗi thành bytes
        input_bytes = user_input.encode('utf-8')

        # Mã hóa Base64
        encoded_bytes = base64.b64encode(input_bytes)

        # Chuyển bytes kết quả thành chuỗi để hiển thị
        encoded_str = encoded_bytes.decode('utf-8')

        print("\n--- Kết quả mã hóa ---")
        print(f"Chuỗi ban đầu: {user_input}")
        print(f"Chuỗi Base64: {encoded_str}")
        print("-----")

    except Exception as e:

```

```

        print(f"Có lỗi xảy ra: {e}")

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab-05\blockchain\block.py`

```

import hashlib
import json

class Block:
    def __init__(self, index, timestamp, transactions, proof, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.transactions = transactions
        self.proof = proof
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        """Tính toán mã băm SHA-256 của Block hiện tại"""
        # Sắp xếp các transaction theo key để đảm bảo tính nhất quán của chuỗi JSON
        block_string = json.dumps({
            'index': self.index,
            'timestamp': self.timestamp,
            'transactions': self.transactions,
            'proof': self.proof,
            'previous_hash': self.previous_hash
        }, sort_keys=True).encode('utf-8')

        return hashlib.sha256(block_string).hexdigest()

    def to_dict(self):
        """Chuyển đổi block thành dictionary phục vụ in ấn"""
        return {
            'index': self.index,
            'timestamp': self.timestamp,
            'transactions': self.transactions,
            'proof': self.proof,
            'previous_hash': self.previous_hash,
            'hash': self.hash
        }

```

Đường dẫn file: `lab-05\blockchain\blockchain.py`

```

import time
import hashlib
from block import Block

class Blockchain:
    def __init__(self):
        self.chain = []
        self.current_transactions = []
        self.difficulty = 4 # Số chữ số 0 đứng đầu yêu cầu khi đào block (độ khó)

        # Tạo Genesis Block (Block đầu tiên của chuỗi)
        self.new_block(proof=100, previous_hash='1')

    def new_block(self, proof, previous_hash=None):
        """Tạo một Block mới và thêm vào chuỗi"""
        block = Block(
            index=len(self.chain) + 1,
            timestamp=time.time(),
            transactions=self.current_transactions,
            proof=proof,
            previous_hash=previous_hash or self.chain[-1].hash
        )

        # Reset danh sách các giao dịch hiện tại
        self.current_transactions = []

        # Thêm block vào chuỗi
        self.chain.append(block)
        return block

    def new_transaction(self, sender, recipient, amount):
        """Thêm giao dịch mới vào danh sách giao dịch chờ đào"""
        self.current_transactions.append({
            'sender': sender,
            'recipient': recipient,
            'amount': amount,
            'timestamp': time.time()
        })
        return self.last_block.index + 1

    @property
    def last_block(self):
        """Lấy Block cuối cùng trong chuỗi"""
        return self.chain[-1]

    def proof_of_work(self, last_block):
        """
        Thuật toán Proof of Work (Mô tả quá trình đào):

```

```

        Tìm một số proof sao cho hash(last_proof, proof, last_hash) có đúng số số 0 đứng đầu
        bằng độ khó.
        """

        last_proof = last_block.proof
        last_hash = last_block.hash
        proof = 0

        while self.valid_proof(last_proof, proof, last_hash) is False:
            proof += 1

        return proof

    def valid_proof(self, last_proof, proof, last_hash):
        """Kiểm tra xem mã băm có thỏa mãn độ khó hay không"""
        guess = f'{last_proof}{proof}{last_hash}'.encode('utf-8')
        guess_hash = hashlib.sha256(guess).hexdigest()
        return guess_hash[:self.difficulty] == '0' * self.difficulty

    def is_chain_valid(self):
        """Kiểm tra tính hợp lệ toàn bộ blockchain"""
        last_block = self.chain[0]
        current_index = 1

        while current_index < len(self.chain):
            block = self.chain[current_index]

            # 1. Kiểm tra Previous Hash có khớp với Hash thực tế của block trước đó không
            if block.previous_hash != last_block.hash:
                print(f"[-] Lỗi tính toán vẹn: Block {block.index} có previous_hash không khớp
với hash của Block {last_block.index}")
                return False

            # 2. Kiểm tra Proof of Work có hợp lệ không
            if not self.valid_proof(last_block.proof, block.proof, last_block.hash):
                print(f"[-] Lỗi Proof of Work: Block {block.index} có proof {block.proof} không
hợp lệ")
                return False

            # 3. Kiểm tra tính đúng đắn của băm nội tại block
            if block.hash != block.calculate_hash():
                print(f"[-] Lỗi băm nội tại: Block {block.index} có dữ liệu bị thay đổi sau khi
tạo")
                return False

            last_block = block
            current_index += 1

        return True

```

Đường dẫn file: `lab-05\blockchain\test\_blockchain.py`

```

import json
from blockchain import Blockchain

def print_block(block):
    print(f"\n===== Block {block.index} =====")
    print(f"Timestamp      : {block.timestamp}")
    print(f"Previous Hash: {block.previous_hash}")
    print(f"Proof           : {block.proof}")
    print(f"Hash            : {block.hash}")
    print("Transactions :")
    if len(block.transactions) == 0:
        print("    (Không có giao dịch - Genesis Block)")
    for tx in block.transactions:
        print(f"        - {tx['sender']} -> {tx['recipient']}: {tx['amount']} coins
({tx['timestamp']})")
    print("=====")

def main():
    print("=== KHIẾM THỬ MÔ PHỎNG BLOCKCHAIN ===")

    # 1. Khởi tạo Blockchain
    print("\nKhởi tạo Blockchain mới...")
    blockchain = Blockchain()

    # In Block đầu tiên (Genesis Block)
    print_block(blockchain.chain[0])

    # 2. Thêm giao dịch và Đào Block 1
    print("\n[+] Thêm giao dịch mới vào Block 1...")
    blockchain.new_transaction(sender="Lê Viết Đức", recipient="Nguyễn Văn A", amount=50.5)
    blockchain.new_transaction(sender="Nguyễn Văn A", recipient="Trần Thị B", amount=10.0)

    print("Đang chạy Proof of Work để tìm Hash hợp lệ (Đang đào)...")
    last_block = blockchain.last_block
    proof = blockchain.proof_of_work(last_block)
    print(f"-> Tìm thấy Proof: {proof}")

    # Tạo Block 1
    block1 = blockchain.new_block(proof)
    print("Đào thành công Block 1!")
    print_block(block1)

    # 3. Thêm giao dịch và Đào Block 2
    print("\n[+] Thêm giao dịch mới vào Block 2...")
    blockchain.new_transaction(sender="Trần Thị B", recipient="Lê Viết Đức", amount=5.0)

    print("Đang chạy Proof of Work (Đang đào)...")
    last_block = blockchain.last_block

```

```

proof = blockchain.proof_of_work(last_block)
print(f"-> Tìm thấy Proof: {proof}")

# Tạo Block 2
block2 = blockchain.new_block(proof)
print("Đào thành công Block 2!")
print_block(block2)

# 4. Kiểm tra tính hợp lệ của Blockchain
print("\n--- Kiểm tra tính hợp lệ của Chuỗi Blockchain ---")
is_valid = blockchain.is_chain_valid()
print(f"Is Blockchain Valid: {is_valid}")
print("-----")

# 5. Thử nghiệm thay đổi dữ liệu trái phép (Tấn công Blockchain)
print("\n[Tấn Công Giả Lập] Thay đổi số tiền giao dịch tại Block 1 từ 50.5 thành 999.0...")
blockchain.chain[1].transactions[0]['amount'] = 999.0

# Tính lại hash xem hệ thống phát hiện không
print("--- Kiểm tra tính hợp lệ sau khi bị thay đổi dữ liệu ---")
is_valid_after_attack = blockchain.is_chain_valid()
print(f"Is Blockchain Valid: {is_valid_after_attack}")
print("-----")

if not is_valid_after_attack:
    print("[SUCCESS] Hệ thống phát hiện dữ liệu Block 1 đã bị giả mạo và từ chối chuỗi này!")
else:
    print("[LỖI] Hệ thống không phát hiện ra việc giả mạo dữ liệu!")

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab-05\img-hidden\decode\_image.py`

```

from PIL import Image
import os

def decode_message_from_image(image_path):
    print(f"Đang đọc ảnh để giải mã từ: {image_path}...")
    if not os.path.exists(image_path):
        print(f"[LỖI] Không tìm thấy ảnh tại: {image_path}")
        return None

    img = Image.open(image_path)
    if img.mode != 'RGB':

```

```

img = img.convert('RGB')

pixels = list(img.getdata())

bin_data = ""
decoded_chars = []

print("Đang tiến hành trích xuất các bit LSB...")

for pixel in pixels:
    r, g, b = pixel

    # Đọc LSB từ R, G, B
    bin_data += str(r & 1)
    bin_data += str(g & 1)
    bin_data += str(b & 1)

    # Gom đủ 8 bits (1 byte) để chuyển đổi thành ký tự
    while len(bin_data) >= 8:
        byte_str = bin_data[:8]
        bin_data = bin_data[8:]

        byte_val = int(byte_str, 2)

        # Nếu gặp ký tự Null (\x00) thì dừng việc giải mã vì đây là dấu hiệu kết thúc
        if byte_val == 0:
            return "".join(decoded_chars)

        try:
            decoded_chars.append(chr(byte_val))
        except Exception:
            # Bỏ qua nếu có ký tự không thể giải mã
            pass

# Nếu đi hết cả ảnh mà không gặp ký tự Null
return "".join(decoded_chars)

def main():
    print("=== GIẤU TIN TRONG ẢNH (DECODE) ===")

    current_dir = os.path.dirname(os.path.abspath(__file__))
    encoded_path = os.path.join(current_dir, "encoded_image.png")
    original_path = os.path.join(current_dir, "input_image.png")

    print("Vui lòng lựa chọn ảnh để tiến hành trích xuất tin:")
    print("1. Giải mã từ ảnh đã giấu tin (encoded_image.png)")
    print("2. Giải mã từ ảnh gốc (input_image.png) để kiểm tra")

    choice = input("Lựa chọn (1 hoặc 2): ").strip()

    if choice == '2':

```

```

        target_path = original_path
        is_original = True
    else:
        target_path = encoded_path
        is_original = False

    decoded_msg = decode_message_from_image(target_path)

    if decoded_msg is not None:
        print("\n--- Kết quả trích xuất ---")
        if is_original:
            print(f"Thông điệp trích xuất từ ảnh gốc: {decoded_msg[:100]}... (Rác dữ liệu)")
            print("\n[GIẢI THÍCH] Tại sao không lấy được thông điệp từ ảnh gốc?")
            print("- Trong ảnh gốc (input_image.png), các bit ít quan trọng nhất (LSB) chứa các chi tiết màu sắc tự nhiên của ảnh.")
            print("- Khi giải mã các bit này, chúng ta chỉ nhận được các giá trị ngẫu nhiên (rác dữ liệu) thay vì thông điệp bí mật.")
            print("- Chỉ khi ảnh đã qua quá trình giấu tin (encoded_image.png), các bit LSB mới được sắp xếp có chủ đích để tạo thành các ký tự của thông điệp.")
        else:
            print(f"Thông điệp bí mật trích xuất được: {decoded_msg}")
            print("-----")

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab-05\img-hidden\encode\_image.py`

```

from PIL import Image
import os

def msg_to_bin(msg):
    """Chuyển đổi chuỗi thông điệp thành chuỗi bit nhị phân (có ký tự Null \x00 kết thúc)"""
    bin_data = []
    for char in msg:
        bin_data.append(format(ord(char), '08b'))
    bin_data.append('00000000') # Ký tự phân tách Null dùng để đánh dấu kết thúc thông điệp khi giải mã
    return ''.join(bin_data)

def encode_message_in_image(image_path, message, output_path):
    print(f"Đang đọc ảnh gốc từ: {image_path}...")
    if not os.path.exists(image_path):
        print(f"[LỖI] Không tìm thấy ảnh đầu vào tại: {image_path}")
        return False

```

```

img = Image.open(image_path)
# Đảm bảo ảnh ở định dạng RGB
if img.mode != 'RGB':
    img = img.convert('RGB')

pixels = list(img.getdata())
width, height = img.size

# Chuyển thông điệp thành nhị phân
bin_message = msg_to_bin(message)
message_len = len(bin_message)

# Kiểm tra xem ảnh có đủ dung lượng chứa thông điệp không
# Mỗi pixel có 3 kênh màu (R, G, B), chứa được tối đa 3 bits
capacity = len(pixels) * 3
if message_len > capacity:
    print(f"[LỖI] Thông điệp quá dài ({message_len} bits). Ảnh chỉ chứa được tối đa {capacity} bits.")
    return False

print(f"Dung lượng ảnh: {capacity} bits. Kích thước thông điệp cần giấu: {message_len} bits.")
print("Đang tiến hành giấu tin vào các bit ít quan trọng nhất (LSB)...")

new_pixels = []
bit_idx = 0

for pixel in pixels:
    r, g, b = pixel

    # Giấu bit vào kênh Red
    if bit_idx < message_len:
        r = (r & ~1) | int(bin_message[bit_idx])
        bit_idx += 1

    # Giấu bit vào kênh Green
    if bit_idx < message_len:
        g = (g & ~1) | int(bin_message[bit_idx])
        bit_idx += 1

    # Giấu bit vào kênh Blue
    if bit_idx < message_len:
        b = (b & ~1) | int(bin_message[bit_idx])
        bit_idx += 1

    new_pixels.append((r, g, b))

# Tạo ảnh mới với dữ liệu pixel đã sửa đổi và lưu lại
new_img = Image.new(img.mode, img.size)
new_img.putdata(new_pixels)
new_img.save(output_path)

```

```

print(f"[SUCCESS] Đã giấu tin thành công và lưu ảnh mới tại: {output_path}")
return True

def main():
    print("=== GIẤU TIN TRONG ẢNH (ENCODE) ===")

    # Thiết lập đường dẫn mặc định
    current_dir = os.path.dirname(os.path.abspath(__file__))
    input_path = os.path.join(current_dir, "input_image.png")
    output_path = os.path.join(current_dir, "encoded_image.png")

    message = input("Nhập thông điệp bí mật muốn giấu vào ảnh: ")
    if not message:
        message = "Day la thong diep mat tu Le Viet Duc 0533"
        print(f"Sử dụng thông điệp mặc định: '{message}'")

    encode_message_in_image(input_path, message, output_path)

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab-05\ssl\client.py`

```

import socket
import ssl
import sys

def main():
    host = '127.0.0.1'
    port = 8443

    print("=== SSL SOCKET CLIENT ===")

    # Tạo SSL Context cho Client
    # Sử dụng Purpose.SERVER_AUTH để xác thực server
    context = ssl.create_default_context(ssl.Purpose.SERVER_AUTH)

    # Bỏ qua xác thực chứng chỉ vì chúng ta dùng chứng chỉ tự ký (Self-signed) cho mục đích thực
    hành
    context.check_hostname = False
    context.verify_mode = ssl.CERT_NONE

    # Tạo socket TCP kết nối đến server
    raw_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:

```

```

# Bọc socket TCP thường thành socket bảo mật SSL/TLS
secure_socket = context.wrap_socket(raw_socket, server_hostname=host)

print(f"Đang kết nối bảo mật tới server tại {host}:{port}...")
secure_socket.connect((host, port))
print("[+] Kết nối và bắt tay SSL/TLS thành công!")

# Nhập tin nhắn gửi lên server
message = input("Nhập tin nhắn để gửi đến Server: ")
if not message:
    message = "Tin nhắn mặc định từ client"

# Gửi dữ liệu đã được mã hóa
secure_socket.sendall(message.encode('utf-8'))
print("[*] Tin nhắn đã được mã hóa và gửi đi.")

# Nhận phản hồi bảo mật từ server
response = secure_socket.recv(1024)
print(f"\n[Server phản hồi]: {response.decode('utf-8')}")

# Đóng kết nối
secure_socket.close()
print("\n[-] Kết nối an toàn đã được đóng.")

except ConnectionRefusedError:
    print("\n[LỖI] Không thể kết nối tới Server. Hãy đảm bảo Server đang chạy trước.")
except Exception as e:
    print(f"\n[LỖI] Có lỗi xảy ra: {e}")

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab-05\ssl\server.py`

```

import socket
import ssl
import os

def main():
    # Cấu hình địa chỉ và cổng
    host = '127.0.0.1'
    port = 8443

    # Lấy đường dẫn tuyệt đối tới file chứng chỉ và khóa riêng
    current_dir = os.path.dirname(os.path.abspath(__file__))
    cert_path = os.path.join(current_dir, 'certificates', 'server.crt')

```

```

key_path = os.path.join(current_dir, 'certificates', 'server.key')

if not os.path.exists(cert_path) or not os.path.exists(key_path):
    print(f"[LỖI] Không tìm thấy file chứng chỉ hoặc khóa riêng.")
    print(f"Vui lòng chạy file certificates/generate_cert.bat trước.")
    return

print("=== SSL SOCKET SERVER ===")

# Tạo SSL Context cho Server
context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
context.load_cert_chain(certfile=cert_path, keyfile=key_path)

# Tạo socket TCP cơ bản
bind_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
bind_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
bind_socket.bind((host, port))
bind_socket.listen(5)

print(f"Server đang lắng nghe kết nối bảo mật tại {host}:{port}...")

try:
    while True:
        # Chấp nhận kết nối TCP từ client
        client_socket, from_address = bind_socket.accept()
        print(f"\n[+] Có kết nối TCP mới từ: {from_address}")

        try:
            # Thiết lập bắt tay SSL/TLS (wrap socket)
            secure_socket = context.wrap_socket(client_socket, server_side=True)
            print("[*] Hoàn tất bắt tay SSL/TLS. Kênh truyền tin đã được mã hóa.")

            # Nhận dữ liệu từ client
            data = secure_socket.recv(1024)
            if data:
                message = data.decode('utf-8')
                print(f"[Client gửi]: {message}")

                # Gửi phản hồi lại cho client
                response = f"Server đã nhận tin nhắn: '{message}' thành công."
                secure_socket.sendall(response.encode('utf-8'))

        except ssl.SSLError as e:
            print(f"[-] Lỗi bảo mật SSL/TLS trong quá trình bắt tay: {e}")
        except Exception as e:
            print(f"[-] Lỗi xử lý kết nối: {e}")
        finally:
            # Đóng kết nối
            secure_socket.close()
            print(f"[-] Đã đóng kết nối an toàn.")

```

```
except KeyboardInterrupt:
    print("\nServer đang dừng hoạt động...")
finally:
    bind_socket.close()
    print("Server đã dừng.")

if __name__ == "__main__":
    main()
```
